XSS If your application re-displays malicious user input from the extranet, the application will be vulnerable to XSS. User names, comments, spam reports, order addresses are just a few uncommon examples, where there can be XSS.
Having one single place in the admin interface or Intranet, where the input has not been sanitized, makes the entire application vulnerable. Possible exploits include stealing the privileged administrator's cookie, injecting an iframe to steal the administrator's password or installing malicious software through browser security holes to take over the administrator's computer.

Refer to the Injection section for countermeasures against XSS. It is *recommended to use the SafeErb plugin* also in an Intranet or administration interface.
CSRF Cross-Site Request Forgery (CSRF), also known as Cross-Site Reference Forgery (XSRF), is a gigantic attack method, it allows the attacker to do everything the administrator or Intranet user may do. As you have already seen above how CSRF works, here are a few examples of what attackers can do in the Intranet or admin interface.
A real-world example is a router reconfiguration by CSRF. The attackers sent a malicious e-mail, with CSRF in it, to Mexican users. The e-mail claimed there was an e-card waiting for them, but it also contained an image tag that resulted in a HTTP-GET request to reconfigure the user's router (which is a popular model in Mexico). The request changed the DNS-settings so that requests to a Mexico-based banking site would be mapped to the attacker's site. Everyone who accessed the banking site through that router saw the attacker's fake web site and had their credentials stolen.
Another example changed Google Adsense's e-mail address and password by. If the victim was logged into Google Adsense, the administration interface for Google advertisements campaigns, an attacker could change their credentials.

Another popular attack is to spam your web application, your blog or forum to propagate malicious XSS. Of course, the attacker has to know the URL structure, but most Rails URLs are quite straightforward or they will be easy to find out, if it is an open-source application's admin interface. The attacker may even do 1,000 lucky guesses by just including malicious IMG-tags which try every possible combination.

For *countermeasures against CSRF in administration interfaces and Intranet applications, refer to the countermeasures in the CSRF section*.

5.1 Additional Precautions

The common admin interface works like this: it's located at www.example.com/admin, may be accessed only if the admin flag is set in the User model, re-displays user input and allows the admin to delete/add/edit whatever data desired. Here are some thoughts about this:

- It is very important to *think about the worst case*: What if someone really got hold of your cookies or user credentials. You could *introduce roles* for the admin interface to limit the possibilities of the attacker. Or how about *special login credentials* for the admin interface, other than the ones used for the public part of the application. Or a *special password for very serious actions*?
- Does the admin really have to access the interface from everywhere in the world? Think about *limiting the login to a bunch of source IP addresses*. Examine request.remote_ip to find out about the user's IP address. This is not bullet-proof, but a great barrier. Remember that there might be a proxy in use, though.
- *Put the admin interface to a special sub-domain* such as admin.application.com and make it a separate application with its own user management. This makes stealing an admin cookie from the usual domain, www.application.com, impossible. This is because of the same origin policy in your